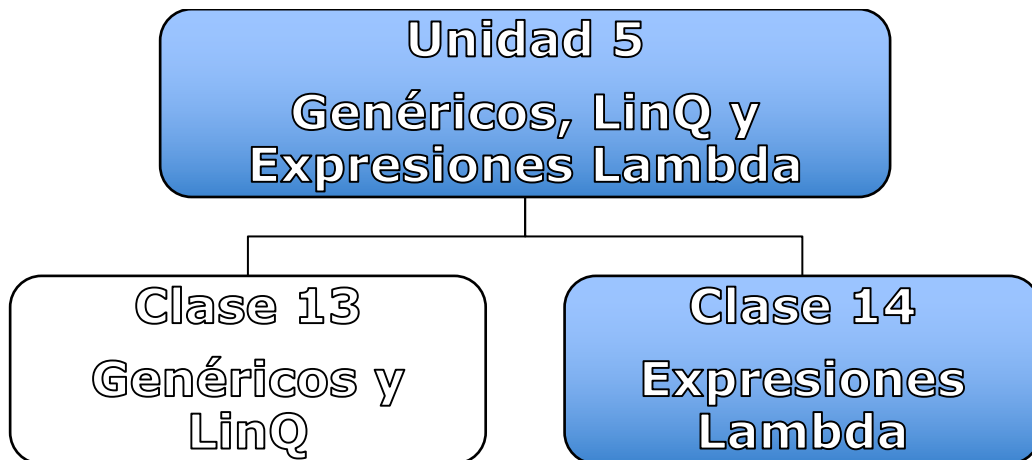

PROGRAMACIÓN ORIENTADA A OBJETOS



Docente titular y autor de contenidos: Prof. Ing. Darío Cardacci

Presentación

En esta unidad veremos todo lo necesario para poder trabajar con Genéricos, linQ y Expresiones Lambda. Esto nos permitirá construir programas más robustos y flexibles.

Por todo lo expresado hasta aquí es que esperamos que usted, a través del estudio de esta unidad, se oriente hacia el logro de las siguientes metas de aprendizaje:

- Reconocer cuando es apropiado utilizar Genéricos para flexibilizar el código desarrollado.
- Contruir consultas con LinQ.
- Mejorar la escritura del código utilizando expresiones Lambda.

Los siguientes **contenidos** conforman el marco teórico y práctico de esta unidad. A partir de ellos lograremos alcanzar el resultado de aprendizaje propuesto. En negrita encontrará lo que trabajaremos en la clase 14.

- Introducción a los genéricos. Ventaja de usar genéricos. Clases genéricas. Interfaces y métodos genéricos.
- LINQ to Object. Introducción a las consultas con LINQ. Escritura de consultas con LINQ. Retorno y almacenamiento de consultas LINQ. Grupos anidados y subconsultas con LINQ.
- **Introducción a las expresiones lambda. Funciones anónimas. Uso de expresiones lambda en consultas.**

A continuación, le presentamos un detalle de los contenidos y actividades que integran esta unidad. Usted deberá ir avanzando en el estudio y profundización de los diferentes temas, realizando las lecturas requeridas y elaborando las actividades propuestas, algunas de desarrollo individual y otras para resolver en colaboración con otros estudiantes y con su profesor tutor.

1. Expresiones Lambda

Lectura requerida

- <https://docs.microsoft.com/en-us/dotnet/csharp/programming-guide/statements-expressions-operators/anonymous-functions>
- <https://docs.microsoft.com/en-us/dotnet/csharp/programming-guide/statements-expressions-operators/lambda-expressions>

Lo/a invitamos ahora a comenzar con el estudio de los contenidos que conforman esta unidad.

1. Expresiones Lambda

Una **expresión lambda** es una función anónima que se puede usar para crear delegados. Al usar **expresiones lambda**, puede escribir funciones locales que se pueden pasar como argumentos, o devolver como el valor de las llamadas a funciones. Las **expresiones Lambda** son particularmente útiles para escribir expresiones de consulta LINQ.

Para crear una **expresión lambda**, especifique los parámetros de entrada (si corresponde) en el lado izquierdo del operador lambda => , y coloque la **expresión o el bloque de instrucciones** en el otro lado. Por ejemplo, la **expresión lambda X => X * X**, especifica un parámetro que se llama X y devuelve el valor de X al cuadrado. Puede asignar esta expresión a un tipo de delegado, como se muestra en el siguiente ejemplo:

```
delegate int del(int i);  
1 referencia  
private void button1_Click(object sender, EventArgs e)  
{  
    del myDelegate = x => x * x;  
    MessageBox.Show(myDelegate(5).ToString());  
}
```

EJ0016

El delegado se puede instanciar como **Func<int,bool> miFunc** donde **int** es un parámetro de entrada y **bool** es el valor de retorno. El valor de retorno siempre se especifica en último lugar, o sea como último parámetro de tipo. **Func<int, string, bool>** define un delegado con dos parámetros de entrada, **int** y **string**, y un tipo de retorno de **bool**. En el siguiente ejemplo **EJ0017**, el delegado **Func**, cuando se invoca, devolverá verdadero o falso para indicar si el parámetro de entrada es igual al valor a comparar:

```
private void button1_Click(object sender, EventArgs e)
{
    Func<int, bool> myFunc = x => x == int.Parse(this.textBox1.Text);
    bool result = myFunc(int.Parse(this.textBox2.Text)); // retorna verdadero o falso
    MessageBox.Show(result.ToString());
}
```

EJ0017

También puede suministrar una expresión lambda cuando el tipo de argumento es un **Expression<Func>**, por ejemplo, en los operadores de consulta estándar definidos en **System.Linq.Queryable**. El ejemplo **EJ0018** expone como se aplica esto en el método **Count** para contar los números impares.

```
private void button1_Click(object sender, EventArgs e)
{
    int[] numbers = { 5, 4, 1, 3, 9, 8, 6, 7, 2, 0 };
    MessageBox.Show(numbers.Count(n => n % 2 == 1).ToString());
}
```

EJ0018

En este ejemplo el compilador puede inferir el tipo del parámetro de entrada, también se puede especificar explícitamente.

El ejemplo **EJ0019** produce una secuencia que contiene todos los elementos de un vector numérico que están en el lado izquierdo del primer valor que es igual o mayor a 7, porque ese es el primer número en la secuencia que no cumple con la condición.

```
private void button1_Click(object sender, EventArgs e)
{
    int[] numeros = { 5, 4, 1, 3, 9, 8, 6, 7, 2, 0 };
    var numerosMenores7 = numeros.TakeWhile(n => n < 7);
    foreach (int n in numerosMenores7)
    {
        this.listBox1.Items.Add(n);
    }
}
```

EJ0019

Se pueden utilizar expresiones Lambda en consultas. El ejemplo **EJ0020** se muestra cómo usar una expresión lambda en una

consulta, utilizando el operador de consulta estándar **Enumerable.Where**. Tenga en cuenta que el método **Where** en este ejemplo, tiene un parámetro de entrada del tipo de delegado Func <TResult> y que el delegado toma un entero como entrada y devuelve un booleano. La **expresión lambda** se puede convertir a ese delegado.

Enumerable.Where, filtra una secuencia de valores basada en un predicado.

Where posee la siguiente firma:

**Where<TSource>(IEnumerable<TSource>,
Func<TSource,Int32,Boolean>)**

En este caso se filtran los números que sean menores o iguales al índice que ocupa en el array multiplicado por 10.

```
1 referencia
private void button1_Click(object sender, EventArgs e)
{
    listBox1.Items.Clear();
    int[] numbers = { 0, 30, 20, 15, 90, 85, 40, 75 };
    IEnumerable<int> query =
        numbers.Where((number, index) => number <= index * 10);

    foreach (int number in query)
    {
        listBox1.Items.Add(number);
    }
}
```

EJ0020

El ejemplo **EJ0021** muestra otra forma de aprovechar el **Where**.

```

1 referencia
private void button1_Click(object sender, EventArgs e)
{
    listBox1.Items.Clear();
    List<string> frutas =
    new List<string> { "manzana", "frutilla", "banana", "mango",
                     "naranja", "uva", "pera", "durazno" };
    IEnumerable<string> consulta = frutas.Where(fruit => fruit.Length < 6);
    foreach (string fruta in consulta)
    {
        listBox1.Items.Add(fruta);
    }
}

```

EJ0021

El ejemplo **EJ0022** muestra como escribir el código del ejemplo **EJ0021** de manera más reducida.

```

1 referencia
private void button1_Click(object sender, EventArgs e)
{
    listBox1.Items.Clear();
    List<string> frutas =
    new List<string> { "manzana", "frutilla", "banana", "mango",
                     "naranja", "uva", "pera", "durazno" };

    foreach (string fruta in frutas.Where(fruit => fruit.Length < 6))
    {
        listBox1.Items.Add(fruta);
    }
}

```

EJ0022